

M6. Héritage et polymorphisme



Ce module se concentre sur l'**approfondissement des concepts d'héritage et de polymorphisme** dans Python 3, des composantes essentielles de la programmation orientée objet. Les participants découvriront comment créer et travailler avec les sous-classes, surcharger et redéfinir les méthodes pour améliorer la fonctionnalité des classes. De plus, le cours explore comment le polymorphisme est utilisé avec les classes abstraites, enrichissant la flexibilité et l'efficacité du code Python. Un choix incontournable pour ceux qui cherchent à optimiser davantage leur maîtrise de Python.

≡ Les concepts d'héritage et de polymorphisme

≡ Création de sous-classes

≡ Surcharge et redéfinition de méthodes

≡ Polymorphisme et classes abstraites

≡ Review

Les concepts d'héritage et de polymorphisme

L'héritage et le polymorphisme sont deux concepts centraux de la POO qui permettent d'organiser son code de manière efficace et modulaire.

L'héritage

L'héritage est un mécanisme qui permet à une classe d'hériter des attributs et des méthodes d'une autre classe, appelée **classe parent** ou classe de base. La classe qui hérite est appelée classe dérivée ou **classe enfant**.

L'héritage permet de créer une hiérarchie de classes, où chaque classe enfant hérite des caractéristiques de sa classe parente, tout en ayant la possibilité d'ajouter ou de modifier ces caractéristiques.

L'héritage favorise la réutilisation du code et en facilite la maintenance.

Par exemple, si nous considérons une classe de base appelée *Animal* avec des attributs et des méthodes communes à tous les animaux, nous pouvons facilement créer des classes dérivées telles que *Chien*, *Chat* et *Oiseau*, qui héritent des attributs et des méthodes de la classe *Animal*.

Chaque classe dérivée peut également ajouter ses propres attributs et méthodes spécifiques. Ainsi, il est possible d'organiser le code de manière hiérarchique et spécialisée.

 La classe **Objet** est la super classe de base de toutes classes en Python.

Le polymorphisme

Le polymorphisme se réfère à la capacité d'un objet d'une classe à prendre différentes formes ou comportements. Cela est généralement réalisé en utilisant des méthodes avec le même nom dans différentes classes, mais avec des implémentations différentes.

Le polymorphisme permet d'utiliser une même interface (nom de méthode) pour effectuer des actions différentes selon le type d'objet auquel elle s'applique.

Prenons l'exemple des classes *Chien*, *Chat* et *Oiseau* mentionnées précédemment.

Chacune de ces classes peut avoir une méthode appelée *faire_du_bruit()*, mais chaque classe l'implémentera de manière différente.

Par exemple, dans la classe *Chien*, *faire_du_bruit()* pourrait retourner "Aboiement!", tandis que dans la classe *Chat*, elle pourrait retourner "Miaulement!" et dans la classe *Oiseau*, elle pourrait retourner "Cancanement!".

Ainsi, en utilisant une même méthode sur différents objets, **le polymorphisme permet d'appeler la version appropriée de la méthode en fonction du type d'objet.**

Le polymorphisme c'est également la capacité d'un objet à être de plusieurs types à la fois.

Dans le cas de notre classe *Chien* par exemple, les types associés à l'objet *Chien* seront :

- *Chien* : le type courant, le type de l'instance que nous avons créé.
- *Animal* : le type du parent.
- *Object* : le type de la super classe de base.

CONTINUER

Création de sous-classes

La création de sous-classes est directement liée au mécanisme d'héritage.

Modélisons notre classe de base *Animal* ?

Nous allons spécifier qu'un animal dispose d'un nom et la capacité de faire du bruit :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def faireDuBruit(self):
        print("L'animal fait du bruit")
```

Script qui définit une classe *Animal*.

Il nous est possible désormais de créer des objets de type *Animal* qui disposeront de leur nom et de la méthode *faireDuBruit()*. Pratique mais insuffisant. En effet, cette classe ne précise en rien de quel animal il s'agit.

Pourquoi dans ce cas ne pas spécialiser notre classe *Animal* au moyen d'une sous classe *Chien*. Ce qui permettrait de spécifier des comportements propres aux chiens :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def faireDuBruit(self):
        print("L'animal fait du bruit")

class Chien(Animal):

    def aboyer(self):
        print("Wouf Wouf!")

monChien = Chien("Bobby")
monChien.faireDuBruit()
monChien.aboyer()
```

Script qui crée une sous-classe Chien.

Nous avons créé une classe *Chien* qui hérite de la classe *Animal*.

Lorsque nous voulons définir une sous classe, vous remarquez qu'une syntaxe particulière est de rigueur :

```
class NomDeLaClasse(NomDuParent)
```

Syntaxe de définition d'une sous-classe.

Cette notation permet de spécifier que notre classe est une classe spécialisée de la classe parent (placée entre parenthèses).

Vous remarquerez peut-être que **notre classe enfant ne dispose pas de constructeur** bien que nous passons le nom "Bobby" lors de la construction de notre objet *Chien*.

En effet, étant donné qu'à ce stade nous n'avons pas d'autre attribut que le nom, il n'est pas utile de préciser un constructeur particulier pour notre classe *Chien*. De ce fait, **le constructeur de la classe parent sera utilisé.**

L'exécution de notre script nous donnera le résultat suivant :

```
L'animal fait du bruit  
Wouf Wouf!
```

Résultat du script précédent.

Nous avons utilisé à la fois la méthode *faireDuBruit()* de la classe parent et la méthode *aboyer()* de la classe enfant.

Que devons-nous faire si nous souhaitons ajouter un attribut *race* à notre classe *Chien*. Il faudra ajouter le paramètre *race* au constructeur de la classe *Chien*, le constructeur de la classe *Animal* ne répondant plus totalement à nos besoins.

Une règle capitale existe lorsque nous devons spécifier un constructeur à une classe enfant : **le constructeur de la classe enfant doit répondre aux besoins de la classe mais également et surtout aux besoins de la classe parent !** Hors de question de créer un constructeur dans la classe *Chien* qui n'intégrerait pas le nom de l'animal.

Voyez plutôt :

```
class Animal:
```

```
def __init__(self, nom):
    self.nom = nom

def faireDuBruit(self):
    print("L'animal fait du bruit")

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def aboyer(self):
        print("Wouf Wouf!")

monChien = Chien("Bobby", "Bouvier Bernois")
monChien.faireDuBruit()
monChien.aboyer()
print(monChien.nom)
print(monChien.race)
```

Script qui ajoute un attribut à une sous-classe.

Nous avons ajouté l'attribut 'race' à notre classe *Chien*.

Ensuite nous n'avons fait rien d'autre qu'utiliser ce que nous avons à notre disposition.

Effectivement, notre classe parent *Animal* dispose du constructeur qui permet d'initialiser l'attribut *nom* de notre objet. Il nous suffit dès lors d'appeler le constructeur de la classe parent au moyen de la **méthode spéciale super()**.

La **méthode super()** permet l'utilisation d'une méthode de la classe parent à partir d'une sous-classe. Elle joue un rôle important lors de la mise en place de l'héritage en évitant la duplication de code et en permettant l'extension de fonctionnalités existantes comme dans notre cas, une extension du constructeur de la classe *Chien*.

L'exécution de notre script nous donnera le résultat suivant :

```
L'animal fait du bruit  
Wouf Wouf!  
Bobby  
Bouvier Bernois
```

Résultat du script précédent.

CONTINUER

Surcharge et redéfinition de méthodes

Nous venons d'aborder le fait de spécialiser le constructeur de la classe *Animal* afin d'avoir un constructeur de la classe *Chien* qui correspond davantage à nos besoins (soit pouvoir ajouter la race du chien à la construction de l'objet).

Ce mécanisme s'appelle la **redéfinition de méthode**.

La redéfinition de méthode consiste en la réécriture au sein d'une classe enfant, d'une méthode présente dans la hiérarchie de la classe (un de ses parents), afin de lui conférer une logique, un traitement, des paramètres différents de la méthode originale.

La redéfinition de méthode est à mettre en parallèle avec **la surcharge de méthode** qui, quant à elle, consiste en la modification du type ou du nombre des paramètres d'une méthode existant dans la classe ou sa hiérarchie, afin de définir un comportement adapté à cette nouvelle situation.

Surcharge de méthode (overloading)

Autant la redéfinition de méthode est correctement implémentée en Python, autant la surcharge de méthode ne l'est malheureusement pas.

Voyez plutôt cet exemple de surcharge :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def faireDuBruit(self):
        print("L'animal fait du bruit")

    def faireDuBruit(self, bruit):
        print(f"L'animal fait du bruit {bruit}")

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def aboyer(self):
        print("Wouf Wouf!")

monChien = Chien("Bobby", "Bouvier Bernois")
monChien.faireDuBruit()
```

Script qui illustre une surcharge de méthode en Python.

Dans cet exemple, nous avons surchargé la méthode *faireDuBruit()* de la classe *Animal* afin de lui permettre d'accepter un paramètre représentant le bruit en question que ferait l'animal.

L'exécution du script nous donne le résultat suivant :

```
TypeError: Animal.faireDuBruit() missing 1 required positional argument: 'l'
```

Résultat du script précédent.

Le système nous renvoie une erreur.

Effectivement, la dernière définition connue de la méthode *faireDuBruit()* attend un paramètre qui n'est pas passé dans notre cas. La version sans paramètre de la méthode est occulté au bénéfice de sa dernière version.

Que faire dans ce cas pour obtenir au moins un semblant de surcharge de méthode ?

Il faut utiliser des paramètres optionnels et en faire en sorte que la méthode adapte son comportement à la présence ou non de ces paramètres.

Réglons le problème de notre méthode *faireDuBruit()* :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def faireDuBruit(self, bruit=None):
        if bruit==None: print("L'animal fait du bruit")
        else: print(f"L'animal fait du bruit {bruit}")

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)
```

```
def aboyer(self):  
    print("Wouf Wouf!")  
  
monChien = Chien("Bobby", "Bouvier Bernois")  
monChien.faireDuBruit()  
monChien.faireDuBruit("Graoouuu")
```

Script qui effectue une surcharge de méthode en Python.

La méthode *faireDuBruit()* a été réécrite en lui permettant d'admettre ou non un paramètre *bruit* dont la valeur par défaut est *None* dans le cas où aucun paramètre n'est passé à la méthode.

Le corps de la méthode a été quant à lui rédigé de telle manière que le comportement de celui-ci sera étroitement lié au fait de recevoir (ou pas) une valeur pour le paramètre.

A l'exécution du code, nous obtiendrons le résultat suivant :

```
L'animal fait du bruit  
L'animal fait du bruit Graoouuu
```

Résultat du script précédent.

Nous avons réussi à implémenter le concept de surcharge de méthode à un langage qui n'a pas prévu de le gérer comme le font bon nombre d'autres langages de programmation.

Remarques importantes :

- Les paramètres optionnels d'une méthode sont déclarés en dernier lieu dans la signature de la méthode.
- Si nous avons plusieurs paramètres optionnels mais que seulement l'un ou l'autre d'entre eux sont utilisés dans notre appel de la méthode, il faudra utiliser les paramètres nommés.

Illustration des remarques :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def faireDuBruit(self, bruit=None, intensite=None):
        if bruit==None and intensite==None: print("L'animal fait du bruit")
        elif intensite==None: print(f"L'animal fait du bruit {bruit}")
        elif bruit==None: print(f"L'animal fait un bruit d'une {intensite}")
        else: print(f"L'animal fait un bruit {bruit} d'une {intensite} intensite")

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def aboyer(self):
        print("Wouf Wouf!")

monChien = Chien("Bobby", "Bouvier Bernois")
monChien.faireDuBruit()
monChien.faireDuBruit("Graou")
monChien.faireDuBruit(intensite="Moyenne")
monChien.faireDuBruit(intensite="Faible", bruit="Miaou")
```

Script qui utilise des paramètres nommés pour utiliser une méthode surchargée.

Nous pouvons remarquer différentes choses dans ce code :

- S'il n'est pas nommé, le premier paramètre passé à la méthode correspondra au premier paramètre de la méthode.
- Si nous souhaitons utiliser des paramètres plus avancés dans l'ordre de déclaration, nous devons impérativement les nommer.
- Si tous les paramètres sont nommés, l'ordre de ceux-ci n'a pas d'importance.

L'exécution du script nous donnera le résultat suivant :

```
L'animal fait du bruit  
L'animal fait du bruit Graou  
L'animal fait un bruit d'une Moyenne intensité  
L'animal fait un bruit Miaou d'une Faible intensité
```

Résultat du script précédent.

Redéfinition de méthode (overriding)

La redéfinition de méthode a déjà été utilisée sans être nommée de la sorte.

En effet, lorsque nous précisons le constructeur de notre classe nous redéfinissons le constructeur de la classe Objet (au plus haut niveau) ou de notre classe parente.

Vous connaissez déjà la méthode `__init__()` qui représente le constructeur de la classe ainsi que la méthode `__del__()` qui représente le destructeur de la classe.

Il existe d'autres méthodes spéciales telles que la méthode `__str__()` qui est la méthode appelée lorsque nous tentons d'afficher notre objet.

Son comportement par défaut est assez peu intéressant, voyez par vous-même ce que ça donne lorsque nous tentons d'afficher un objet:

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

monChien = Chien("Bobby", "Bouvier Bernois")
print(monChien)
```

Script qui affiche un objet.

Comme précisé, ce résultat n'a que peu d'intérêt. L'affichage par défaut de l'objet consiste en son script de provenance, la classe associée ainsi qu'une référence mémoire de l'instance de l'objet:


```
<__main__.Chien object at 0x7f69442ff2b0>
```

Résultat du script précédent.

Si par contre, nous redéfinissons la méthode `__str__()` nous pouvons lui faire dire ce que nous voulons comme par exemple les informations associées à l'objet :

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

    def __str__(self):
        return f"Je suis un objet de la classe animal avec le nom {self.nom}"

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def __str__(self):
        return super().__str__() + f"\nplus particulièrement un chien de race {self.race}"

monChien = Chien("Bobby", "Bouvier Bernois")
print(monChien)
```

Script qui redéfinit la méthode `__str__`.

L'exécution de ce script nous produira un résultat nettement plus satisfaisant :

```
Je suis un objet de la classe animal avec le nom Bobby  
plus particulièrement un chien de race Bouvier Bernois
```

Résultat du script précédent.

En effet, lors de la tentative d'affichage de l'objet, le système va chercher à appeler implicitement la méthode `__str__()` se situant au plus près de l'instance de la classe courante. En réalité c'est la version de `__str__()` enregistrée en mémoire lors de l'instanciation de l'objet qui est utilisée.

Notre objet étant de type *Chien* et notre classe *Chien* redéfinissant la méthode `__str__()` c'est celle-ci qui sera utilisée lors de l'affichage de l'objet.

CONTINUER

Polymorphisme et classes abstraites

Compléments sur le polymorphisme

Un point intéressant du polymorphisme est la capacité pour un objet de pouvoir « revêtir » plusieurs types différents.

Ces types sont implicites à l'objet courant dès l'instant où ils sont présents dans la hiérarchie de l'objet.

Par exemple, tout objet Python possède au minimum 2 types :

- Le type de la classe d'appartenance.
- Le type du super-parent de chaque objet en Python à savoir le type *Object*.

Il existe une fonction très pratique qui permet de **déterminer si un objet appartient à tel ou tel autre type**, la méthode **`isinstance()`**.

Cette méthode répond au modèle de code suivant :

```
isinstance(objetAEvaluer, TypeAComparer)
```

Syntaxe de la méthode `isinstance()`.

Cette méthode renverra *True* si l'objet évalué est du type spécifié, *False* dans le cas contraire.

Reprenons le cas de notre classe *Animal* et évaluons l'appartenance de notre objet de type *Chien*:

```
class Animal:

    def __init__(self, nom):
        self.nom = nom

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

monChien = Chien("Bobby", "Bouvier Bernois")

print(f"monChien est de type Chien : {isinstance(monChien, Chien)}")
print(f"monChien est de type Animal : {isinstance(monChien, Animal)}")
print(f"monChien est de type Objet : {isinstance(monChien, object)}")
print(f"monChien est de type List : {isinstance(monChien, list)}")
```

Script qui vérifie l'appartenance d'un objet à une classe.

Dans ce script et grâce à l'utilisation de la méthode *isinstance()*, nous pouvons déterminer la ou les classes d'appartenance de notre objet:

```
monChien est de type Chien : True  
monChien est de type Animal : True  
monChien est de type Objet : True  
monChien est de type List : False
```

Résultat du script précédent.

Dans le mécanisme d'héritage, on peut déterminer avec certitude les faits suivants :

- Tout type d'un objet appartenant à une sous classe peut être de son propre type et de tous les types de son/ses parents.
- Tout type d'un objet appartenant à une classe parente ne peut pas forcément être de tous les types de son/ses enfants (le type d'appartenance sera à vérifier avant toute manipulation de l'objet sous ce type).

Pour reprendre l'exemple de nos classes *Animal* et *Chien* :

On peut dire avec certitude que tout objet de type *Chien* est de type *Animal* mais on ne peut pas être affirmatif quant au fait que tout objet de type *Animal* est de type *Chien*.

 De la même manière qu'un enfant partage les attributs et méthode de son/ses parents, un parent n'aura aucun accès aux méthodes et attributs de ses enfants.

Les classes abstraites

Les classes abstraites sont des classes qui contiennent au minimum une méthode abstraite et qui héritent de la classe `AbstractBaseClass` (ABC).

Les méthodes abstraites quant à elles sont des méthodes qui ne possèdent pas de corps et qui sont déclarées au moyen du décorateur `@abstractmethod`.

La déclaration d'une classe abstraite passe donc à la fois par l'utilisation de la classe **ABC** et du décorateur **`@abstractmethod`**.

La particularité d'une classe abstraite est qu'étant donné que cette classe dispose de méthodes non implémentées, **elle ne peut pas être instanciée**.

Ce comportement est extrêmement intéressant lorsque nous voulons mettre en place des classes permettant de regrouper un ensemble d'attributs et de méthodes sans pour autant en définir le comportement.

C'est la spécialisation de cette classe abstraite qui fournira le comportement de la ou des méthodes abstraites.

Dans le cadre de notre classe *Animal*, il n'est que peu intéressant de définir tous les comportements (par exemple, une méthode *crier()*) car chaque classe spécialisée (telle que la classe *Chien*) ira définir le comportement de sa propre à la classe.

Cependant, le fait de définir la méthode abstraite au sein de la classe (abstraite par extension) permet d'assurer à chaque classe enfant de définir ladite méthode et le comportement qui lui est dédié.

Ainsi chaque classe enfant disposera de la même méthode mais avec le comportement adéquat, rattaché à la classe concernée.

Reprenons l'exemple de nos classes *Animal* et *Chien* et ajoutons-y la classe *Chat* :

```
from abc import ABC, abstractmethod

class Animal(ABC):

    def __init__(self, nom):
        self.nom = nom

    @abstractmethod
    def crier(self):
        pass

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def crier(self):
        return "Le chien fait wouf wouf"

class Chat(Animal):

    def crier(self):
        return "Le chat fait miaaaaaaou"

monChien = Chien("Bobby", "Bouvier Bernois")
monChat = Chat("Garfield")
print(monChien.crier())
print(monChat.crier())
```

Script qui utilise une classe abstraite.

Dans ce script, une classe abstraite *Animal* définit une méthode abstraite *crier()* qui n'implémente pas de corps.

Les classes *Chien* et *Chat* héritent de la classe abstraite et, en tant que classes concrètes, ont l'obligation de définir le corps de la méthode abstraite de son parent. Ce qui a été réalisé.

L'exécution de notre script nous donne le résultat suivant :

```
Le chien fait wouf wouf
Le chat fait miaaaaaaou
```

Résultat du script précédent.

Comme expliqué, une classe abstraite ne peut pas être instanciée, voyons ce qui se passe si nous tentons de le faire :

```
from abc import ABC, abstractmethod

class Animal(ABC):

    def __init__(self, nom):
        self.nom = nom

    @abstractmethod
    def crier(self):
        pass

class Chien(Animal):

    def __init__(self, nom, race):
        self.race = race
        super().__init__(nom)

    def crier(self):
        return "Le chien fait wouf wouf"

class Chat(Animal):

    def crier(self):
```



```
        return "Le chat fait miaaaaaaou"

monAnimal = Animal("Jackal")
```

Tentative d'instanciation d'une classe abstraite.

Le résultat du script sera le suivant :

```
TypeError: Can't instantiate abstract class Animal with abstract method cr:
```

Résultat du script précédent.



Ce résultat sera identique pour toute tentative d'instanciation d'une classe qui n'implémente pas les membres abstraits de son parent.

CONTINUER

Review

Vrai ou faux.

La classe de base de tout objet en python est la classe Object.

☐

Vrai

☐

Faux

SUBMIT

Qu'est ce qu'une classe enfant ?

☐

Une classe qui a hérité d'une autre classe.



Une classe qui est utilisée par une autre classe à des fins de spécialisation.

SUBMIT

Complétez la phrase suivante :

La capacité d'un objet d'une classe à prendre différentes formes ou comportements est appelé le _____ .

Tapez votre réponse ici

SUBMIT

Quelle fonction permet de déterminer l'appartenance d'un objet par rapport à un type spécifique ?

☐ `isinstance()`

☐ `istype()`

☐ `typeof()`

SUBMIT

De quoi avons-nous impérativement besoin pour déclarer une classe abstraite ?

☐ De faire dériver la classe de la classe `AbstractBaseClass(ABC)`.

☐ De l'annotation `@abstractmethod`.

☐ De définir au moins une méthode abstraite au sein de la classe qu'on veut abstraire.

SUBMIT

CONTINUER